Εισαγωγή στη Γονιδιωματική Ανάλυση με το Bioconductor

Πρακτικός οδηγός Hands-on για το Workshop: βασικές δομές S4, γονιδιωματικές συντεταγμένες, ανάλυση αλληλουχιών και ποιοτικός έλεγχος NGS δεδομένων σε R.

Εισηγητής Workshop: Νικόλαος Κατσικαβάλης (nkatsikavalis@uth.gr) | Τμήμα Πληροφορικής με Εφαρμογές στη Βιοϊατρική, Πανεπιστήμιο Θεσσαλίας

Τα Δεδομένα & Αρχεία του Σεμιναρίου

- **Αρχεία FASTA (`mock_virus.fa`):** Ψηφιακό κείμενο με αλληλουχίες βάσεων DNA (A, T, C, G) για απλή ανάγνωση αλληλουχιών.
- **Αρχεία FASTQ (`sample_1.fq`, `sample_2.fq`):** Ακατέργαστες αναγνώσεις από μηχανήματα αλληλούχισης (NGS). Περιέχουν τόσο τις βάσεις όσο και βαθμούς αξιοπιστίας/ποιότητας (Phred scores).
- **Γονιδίωμα Αναφοράς (`BSgenome...sacCer3`):** Ολόκληρη η αλληλουχία του γονιδιώματος της μαγιάς (σακχαρομύκητας) αποθηκευμένη τοπικά.
- **Βάση Σχολιασμού (`TxDb...hg38`):** Ο «χάρτης» του ανθρώπινου γονιδιώματος με τις ακριβείς θέσεις γονιδίων, μεταγραφημάτων και εξονίων.

Προετοιμασία Περιβάλλοντος & Mock Δεδομένα (Setup)

Για τις ανάγκες του workshop δημιουργούμε αυτόματα έναν τοπικό φάκελο `data/` με ελαφριά mock αρχεία FASTA και FASTQ. Έτσι εξασφαλίζουμε άμεση εκτέλεση χωρίς να απαιτείται λήψη αρχείων μεγάλου όγκου.

```
# Ορισμός φακέλου και δημιουργία mock δεδομένων
if (!dir.exists("data")) dir.create("data")

message("Δημιουργία δεδομένων...")
fasta_content <- c(">seq1", "ATGCGTACGTAGCTAG", ">seq2", "CGTACGTAGCTAGCTA")
writeLines(fasta_content, "data/mock_virus.fa")

fastq1_content <- c("@SEQ1", "GATTTGGGGTTCAAAG", "+", "!''*((((***+))%%",
                    "@SEQ2", "ATCGTACGATCGATCG", "+", "IIIIIIIIIIIIIIII")
writeLines(fastq1_content, "data/sample_1.fq")

fastq2_content <- c("@SEQ1_S2", "GATTTGGGGTTCAAAG", "+", "!''*((((***+))%%",
                    "@SEQ2_S2", "ATCGTACGATCGATCG", "+", "IIIIIIIIIII####")
writeLines(fastq2_content, "data/sample_2.fq")

message("Εγκατάσταση πακέτων (Αυτό θα πάρει λίγα λεπτά)...")
if (!requireNamespace("BiocManager", quietly = TRUE)) install.packages("BiocManager")
BiocManager::install(c(
  "Biostrings",
  "GenomicRanges",
  "Rqc",
  "BSgenome.Scerevisiae.UCSC.sacCer3",
  "TxDb.Hsapiens.UCSC.hg38.knownGene"
), update = FALSE, ask = FALSE)
message("Όλα έτοιμα!")
```

ΑΝΑΜΕΝΟΜΕΝΟ OUTPUT:

```
Δημιουργία δεδομένων...
Εγκατάσταση πακέτων (Αυτό θα πάρει λίγα λεπτά)...
Όλα έτοιμα!
```

Ενότητα 1: Έλεγχος Περιβάλλοντος & Εγκυρότητας (Hands-on)

Τι κάνουμε εδώ: Επαληθεύουμε ότι το σύστημα κάθε συμμετέχοντα έχει ρυθμιστεί σωστά και ότι η έκδοση του Bioconductor είναι συγχρονισμένη με την R.

```
library(BiocManager)
BiocManager::version()
```

```
BiocManager::valid()
sessionInfo()
```

ANAMENOMENO OUTPUT:

```
> BiocManager::version()
[1] '3.23'

> BiocManager::valid()
Bioconductor version '3.23'
- 7 packages out-of-date
- 0 packages too new
Warning message:
7 packages out-of-date; 0 packages too new
```

Ενότητα 2: Η Αρχιτεκτονική S4 στο Bioconductor (Hands-on)

Τι κάνουμε εδώ: Εξετάζουμε πώς το Bioconductor αποθηκεύει δεδομένα με ασφάλεια μέσω προκαθορισμένων πεδίων ("slots"). Δημιουργούμε ένα S4 `DataFrame` (με κεφαλαίο D) που υποστηρίζει σύνθετες γονιδιωματικές δομές.

```
library(S4Vectors)

# Δημιουργία ενός DataFrame (αντικείμενο S4) με δεδομένα
df <- DataFrame(chromosome = c("chr1", "chr2"), genes = c(10, 15))

# Έλεγχος αν το αντικείμενο ανήκει στην αρχιτεκτονική S4
isS4(df)

# Εμφάνιση των διαθέσιμων μεθόδων για την κλάση DataFrame
showMethods(class = "DataFrame")
```

ANAMENOMENO OUTPUT:

```
> isS4(df)
[1] TRUE

> showMethods(class = "DataFrame")
Function: [ (package base)
x="DataFrame"

Function: [[ (package base)
x="DataFrame"

Function: as.data.frame (package BiocGenerics)
x="DataFrame"

Function: cbind (package BiocGenerics)
...="DataFrame"

Function: nrow (package base)
x="DataFrame"

Function: show (package methods)
object="DataFrame"
```

Ενότητα 3: Διαχείριση Βιολογικών Αλληλουχιών (Biostrings) (Hands-on)

Τι κάνουμε εδώ: Χειριζόμαστε αλληλουχίες DNA/RNA, υπολογίζουμε το συμπληρωματικό αντίστροφο, μεταφράζουμε σε πρωτεΐνη και διαβάζουμε αρχεία FASTA.

```
library(Biostrings)

# Ορισμός αλληλουχίας DNA
dna_seq <- DNAString("ATGCGTACGTAGCTAGCTAGT")

# Μεταγραφή σε RNA και μετάφραση σε πρωτεΐνη
rna_seq <- RNAString(dna_seq)
rna_seq

protein_seq <- translate(dna_seq)
```

```
protein_seq

# Συμπληρωματικότητα και αντίστροφη φορά (5' -> 3')
reverseComplement(dna_seq)

# Ανάγνωση αρχείου FASTA από τον φάκελο data
fasta_reads <- readDNAStringSet("data/mock_virus.fa")
width(fasta_reads)
```

Βασικές Έννοιες:

- `translate()`: Μεταφράζει τις τριπλέτες βάσεων (κωδικόνια) σε αμινοξέα (ο αστερίσκος * δηλώνει κωδικόνιο λήξης / Stop codon).
- `reverseComplement()`: Αντιστρέφει τη φορά και αλλάζει τις βάσεις στις συμπληρωματικές τους (A ↔ T, C ↔ G).

ANAMENOMENO OUTPUT:

```
> rna_seq
21-letter RNAString object
seq: AUGCGUACGUAGCUAGCUAGU

> protein_seq
7-letter AAString object
seq: MRT*LAS

> reverseComplement(dna_seq)
21-letter DNAString object
seq: ACTAGCTAGCTACGTACGCAT

> width(fasta_reads)
[1] 16 16
```

Ενότητα 4: Γονιδιώματα Αναφοράς με το BSgenome (Hands-on)

Τι κάνουμε εδώ: Φορτώνουμε το πλήρες γονιδίωμα του σακχαρομύκητα (μαγιά) και αντλούμε συντεταγμένες και υπο-αλληλουχίες.

```
library(BSgenome.Scerevisiae.UCSC.sacCer3)

# Ανάθεση του γονιδιώματος σε μεταβλητή
yeast <- BSgenome.Scerevisiae.UCSC.sacCer3

# Επισκόπηση χρωμοσωμάτων και μεγεθών
seqlengths(yeast)

# Εξαγωγή μιτοχονδριακής αλληλουχίας (chrM, βάσεις 1 έως 50)
chrM_sub <- getSeq(yeast, names = "chrM", start = 1, end = 50)
chrM_sub
```

ANAMENOMENO OUTPUT:

```
> seqlengths(yeast)
   chrI   chrII  chrIII   chrIV    chrV   chrVI  chrVII chrVIII   chrIX    chrX
 230218  813184  316620 1531933  576874  270161 1090940  562643  439888  745751
  chrXI  chrXII chrXIII  chrXIV   chrXV  chrXVI    chrM
 666816 1078177  924431  784333 1091291  948066   85779

> chrM_sub
50-letter DNAString object
seq: TTCATAATTAATTTTTTATATATATATTATATTATAATATTAATTTATAT
```

Ενότητα 5: Γονιδιωματικά Διαστήματα (IRanges & GRanges) (Hands-on)

Τι κάνουμε εδώ: Μαθαίνουμε πώς ορίζονται περιοχές στο γονιδίωμα (1-based closed intervals). Ξεχωρίζουμε τα απλά διαστήματα (`IRanges`) από τις πλήρεις συντεταγμένες με χρωμόσωμα και αλυσίδα (`GRanges`).

Φορά Αλυσίδας (Strand): (+) Θετική αλυσίδα 5' → 3' | (-) Αρνητική συμπληρωματική 3' → 5'

```
library(IRanges)
library(GenomicRanges)

# Βασικά διαστήματα μήκους (IRanges)
ir <- IRanges(start = c(1, 100), end = c(50, 250))
ir

# Γονιδιωματικά διαστήματα με χρωμόσωμα και αλυσίδα (GRanges)
gr <- GRanges(
  seqnames = c("chr1", "chr2"),
  ranges = IRanges(start = c(100, 500), end = c(300, 750)),
  strand = c("+", "-")
)
gr
```

ANAMENOMENO OUTPUT:

```
> ir
IRanges object with 2 ranges and 0 metadata columns:
          start       end     width
      <integer> <integer> <integer>
  [1]         1        50        50
  [2]       100       250       151

> gr
GRanges object with 2 ranges and 0 metadata columns:
      seqnames    ranges strand
         <Rle> <IRanges>  <Rle>
  [1]     chr1   100-300      +
  [2]     chr2   500-750      -
  -------
  seqinfo: 2 sequences from an unspecified genome; no seqlengths
```

Ενότητα 6: Πράξεις Επικάλυψης σε Διαστήματα (Overlaps) (Hands-on)

Τι κάνουμε εδώ: Ελέγχουμε αν δύο πειραματικά ευρήματα επικαλύπτονται χωρικά — βρίσκουμε αν οι περιοχές που ανακαλύψαμε συμπίπτουν με γνωστές περιοχές αναφοράς.

End = Start + Width - 1

```
# Ορισμός περιοχών (start & width)
# Query 1: 50 έως (50+60-1) = 50-109
# Query 2: 150 έως (150+60-1) = 150-209
query <- GRanges("chr1", IRanges(start = c(50, 150), width = 60))

# Subject 1: 100 έως (100+80-1) = 100-179
# Subject 2: 200 έως (200+80-1) = 200-279
subject <- GRanges("chr1", IRanges(start = c(100, 200), width = 80))

# Εντοπισμός επικαλύψεων
hits <- findOverlaps(query, subject)
hits

# Φιλτράρισμα περιοχών με επικάλυψη
subsetByOverlaps(query, subject)
```

Ανάλυση Επικαλύψεων:

- `Query 1 (50-109)` επικαλύπτεται με το `Subject 1 (100-179)` στο διάστημα 100-109.
- `Query 2 (150-209)` επικαλύπτεται με το `Subject 1 (100-179)` στο διάστημα 150-179.
- `Query 2 (150-209)` επικαλύπτεται με το `Subject 2 (200-279)` στο διάστημα 200-209.

ANAMENOMENO OUTPUT:

```
> hits
Hits object with 3 hits and 0 metadata columns:
      queryHits subjectHits
      <integer>   <integer>
  [1]         1           1
  [2]         2           1
```

```
  [3]         2           2
  -------
  queryLength: 2 / subjectLength: 2

> subsetByOverlaps(query, subject)
GRanges object with 2 ranges and 0 metadata columns:
      seqnames    ranges strand
         <Rle> <IRanges>  <Rle>
  [1]     chr1    50-109      *
  [2]     chr1   150-209      *
  -------
  seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

Ενότητα 7: Βάσεις Σχολιασμού Γονιδιώματος (TxDb) (Hands-on)

Τι κάνουμε εδώ: Εξάγουμε πληροφορίες για γονίδια και εξόνια του ανθρώπου από επίσημες βάσεις σχολιασμού (UCSC hg38), εστιάζοντας στο χρωμόσωμα X.

```
library(TxDb.Hsapiens.UCSC.hg38.knownGene)

txdb <- TxDb.Hsapiens.UCSC.hg38.knownGene

# Περιορισμός της ανάλυσης στο χρωμόσωμα X
seqlevels(txdb) <- "chrX"

# Εξαγωγή εξονίων ανά μετάγραφημα
exons_per_tx <- exonsBy(txdb, by = "tx")
exons_per_tx[1:2]
```

ANAMENOMENO OUTPUT:

```
> exons_per_tx[1:2]
GRangesList object of length 2:
$`372516`
GRanges object with 2 ranges and 3 metadata columns:
      seqnames        ranges strand |   exon_id   exon_name exon_rank
         <Rle>     <IRanges>  <Rle> | <integer> <character> <integer>
  [1]     chrX 253743-253846      + |    864905        <NA>         1
  [2]     chrX 254937-255091      + |    864906        <NA>         2
  -------
  seqinfo: 1 sequence from hg38 genome

$`372517`
GRanges object with 8 ranges and 3 metadata columns:
      seqnames        ranges strand |   exon_id   exon_name exon_rank
  [1]     chrX 276322-276394      + |    864907        <NA>         1
  [2]     chrX 281482-281684      + |    864913        <NA>         2
  [3]     chrX 284167-284314      + |    864918        <NA>         3
  [4]     chrX 288733-288869      + |    864921        <NA>         4
  [5]     chrX 290648-290776      + |    864923        <NA>         5
  [6]     chrX 291499-291654      + |    864928        <NA>         6
  [7]     chrX 293035-293218      + |    864929        <NA>         7
  [8]     chrX 299097-303353      + |    864930        <NA>         8
  -------
  seqinfo: 1 sequence from hg38 genome
```

Ενότητα 8: Ανάγνωση & Φιλτράρισμα Δεδομένων NGS (ShortRead)

Τι κάνουμε εδώ: Διαβάζουμε αρχεία FASTQ, βλέπουμε ποιότητες ανάγνωσης (Phred scores σε κωδικοποίηση ASCII) και φιλτράρουμε ώστε να κρατήσουμε μόνο τα αξιόπιστα reads.

```
library(ShortRead)

# Ανάγνωση αρχείου FASTQ
fq <- readFastq("data/sample_1.fq")

sread(fq)
quality(fq)

# Δημιουργία φίλτρου ελάχιστου μήκους (>= 16 βάσεις)
len_filter <- srFilter(function(x) width(x) >= 16, name = "MinLength16")
```

```
fq_filtered <- fq[len_filter(fq)]
fq_filtered

length(fq_filtered)
```

Ερμηνεία Εξόδου & Phred Scores:

- **width: 16 cycles:** Το μηχάνημα αλληλούχισης εκτέλεσε 16 χημικούς κύκλους ανάγνωσης (reads μήκους 16 βάσεων).
- **! :** Χαμηλή εμπιστοσύνη ανάγνωσης βάσης (Phred score 0).
- **I :** Εξαιρετικά υψηλή ποιότητα ανάγνωσης (Phred score 40, πιθανότητα σφάλματος 1 στις 10.000).

ANAMENOMENO OUTPUT:

```
> sread(fq)
DNAStringSet object of length 2:
    width seq
[1]    16 GATTTGGGGTTCAAAG
[2]    16 ATCGTACGATCGATCG

> quality(fq)
class: FastqQuality
quality:
BStringSet object of length 2:
    width seq
[1]    16 !''*((((***+))%%
[2]    16 IIIIIIIIIIIIIIII

> fq_filtered
class: ShortReadQ
length: 2 reads; width: 16 cycles

> length(fq_filtered)
[1] 2
```

Ενότητα 9: Αυτοματοποιημένος Ποιοτικός Έλεγχος (Rqc)

Τι κάνουμε εδώ: Μαζικός ποιοτικός έλεγχος αρχείων αλληλούχισης στον φάκελο data και δημιουργία γραφήματος Boxplot για τη σύγκριση της ποιότητας ανά δείγμα.

```
# Φόρτωση του πακέτου Rqc για ποιοτικό έλεγχο
library(Rqc)

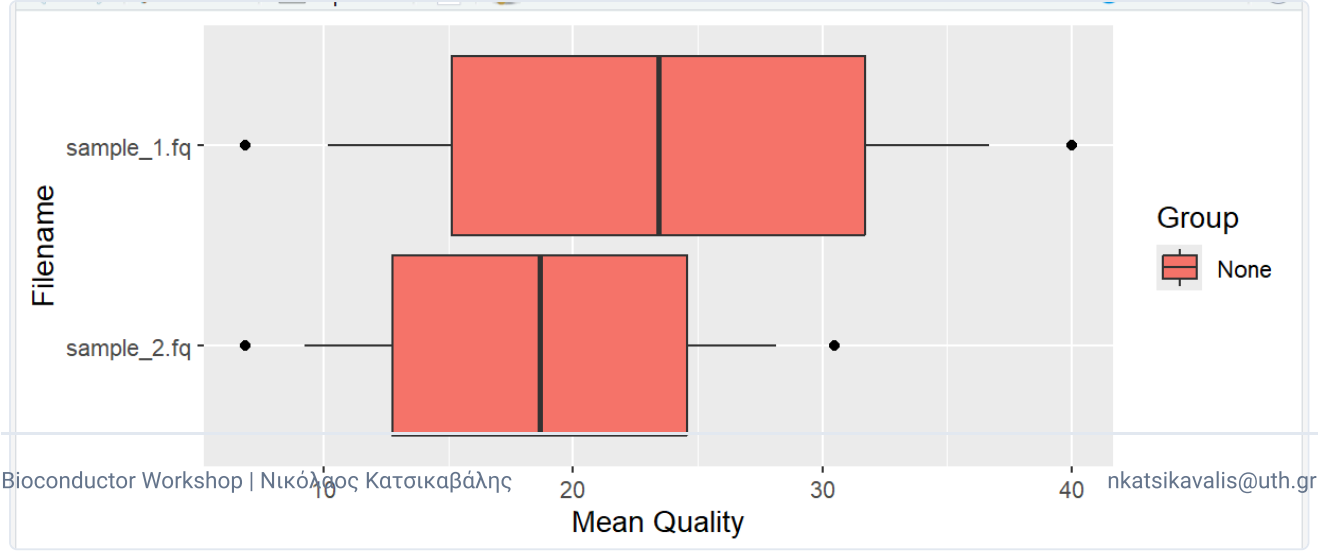
# Αυτόματη σάρωση του φακέλου data για αρχεία .fq
qc_results <- rqc(path = "data", pattern = ".fq$", sample = TRUE)

# Δημιουργία γραφήματος boxplot συνολικής ποιότητας ανά δείγμα
rqcReadQualityBoxPlot(qc_results)
```

ANAMENOMENO OUTPUT:

```
'C:\Users\...\AppData\Local\Temp\...\rqc_report.html' has been created.
```

Οπτικοποίηση Ποιότητας (Boxplot)



Σχήμα: Κατανομή μέσης ποιότητας (Mean Quality) ανά δείγμα FASTQ.

Ερμηνεία Διαγράμματος:

- **sample_1.fq:** Παρουσιάζει υψηλότερη μέση ποιότητα ανάγνωσης (διάμεσος γύρω στο Q24, με ανώτερες τιμές έως και Q40), αλλά με μεγαλύτερη διακύμανση λόγω των χαμηλότερων πρώτων βάσεων (! ' ' *).
- **sample_2.fq:** Εμφανίζει χαμηλότερη συνολική κατανομή ποιότητας (διάμεσος γύρω στο Q19), εξαιτίας της πτώσης ποιότητας στο τέλος των reads (####).